Signed, Complements, and Floating Number

Overview

- Binary vs. BCD representation
- Signed integers
- Sign-and-magnitude representation
- Complementary representations
- Addition and subtraction
- Floating point
- IEEE 754

Unsigned integers (reminder)

- Unsigned representation – just store any whole number in its full representation.
  - Unsigned 4-bit bindary rangfing from 0000 to 1111 represent decimal value raning from 0 to 15
  - Unsigned 8-bit hexadecimal 0x00 ~ 0xFF represents decimal value raning from 0 to 255.

Unsigned integers: BCD

- **Binary-coded decimal** (BCD):
  - Digit-by-digit binary representation of original decimal integer.
- Each decimal digit is **individually** converted to binary.
  - This requires **4 bits per digit** (not all 4-bits patterns are used).
- Example.
  - Decimal value 68 is presented in BCD as 01101000.
- One byte can store **unsigned integers** in a range 0-99 under BCD encoding (Why?).

Binary vs. BCD representation

- BCD is less economical than binary representation (Why?).
- Calculations in BCD are more difficult.
- But, translation between BCD and character form is easier.
  - Last 4 bits of ASCII numeric character codes correspond precisely to the BCD representation.
- Example.
  - ASCII code of ‘5’ is 0110101, its BCD representation is 0101.
- Binary representation is more common, BCD is used for some business applications.

Signed integers

- Several ways of representing negative numbers
  - A simple way is to add a sign bit to the binary integer
    - 0 stands for +
    - 1 stands for -.

Sign-and-magnitude

- 1 sign bit for 4-bit bindary

Decimal	Sign	Binary Magnitude	Full Number
+1	0	0001	00001
-1	1	0001	10001
+5	0	0101	00101
-5	1	0101	10101
+0	0	0000	00000
-0	1	0000	10000

Discussion

- What is the maximum integer of an 8-bit unsigned binary.
- What is the maximum positive of an 8-bit signed binary represented in the above simple way
- What is the minimum negative of an 8-bit signed binary represented in the above simple way

Addition of Sign-and-magnitude

- Example 1 - Signs are the same
  - Just add the magnitudes

```
  0 0011  (+3)
+ 0 0100  (+4)
  ---------
  0 0111  (+7)
```

Addition of Sign-and-magnitude (cont)

- Example 2 - Signs are different
  - first determine which is larger magnitude
  - Put larger magnitude number on top, subtract
  - Result has sign of larger magnitude number…

```
  1 010
- 0 001
-------
  1 001
```

Addition of Sign-and-magnitude (cont)

- Addition requires two separate operations
  - addition
  - subtraction
- Several decisions to make:
  - Signs same or different?
  - Which operand is larger?
  - What is sign of final result?
- Hard to implement in logic (in week 10)
- Two zeros

One's Complement

- Positive numbers are the same as sign-magnitude
- Negative number, such as -N, is represented as the complement of +N: -N = N′

Number	1's Complement
+1	0001
-1	1110
+5	0101
-5	1010
+0	0000
-0	1111

One's Complement Example

- What are the decimal value of
  - 6-bit one's complement 000101
  - 6-bit one's complement 010100
  - 6-bit one's complement 110101
  - 4-bit one's complement 0101
  - 4-bit one's complement 1100

One's Complement Example

– If there is a carry-out on the left, it is wrapped around and added back in (end-around carry)

```
  000101   (5)
+ 010100   (20)
 ----------------
  011001   (25)
```

```
   0101
+  1100
 -------
  1 0001
      + 1   ← end-around
carry
 -------
   0010   (= 5 + (–3) = 2)
```

One's Complement Example

– Now it is your turn to calculte the sum of 2 one's complement 1010 + 1101

One's complement features

- Addition requires no pre-calculation decisions (unlike Sign-Magnitude)
- Addition requires end-around carry
- Still two zeroes (+0, -0)

Two's Complement

- Treat positional digits differently
  - Most significant bit (MSB) given negative weight
  - Other bits same as in unsigned

$0111 = -1 \times 0 \times 2^3 + 1 \times 2^2 + 1 \times 2^1 + 1 \times 2^0 = 7$

$1111 = -1 \times 1 \times 2^3 + 1 \times 2^2 + 1 \times 2^1 + 1 \times 2^0 = -1$

Two's Complement

- More examples

Number	2's Complement
+1	0001
-1	1111
+5	0101
-5	1011
+0	0000
-0	none

One's and Two's Complement

- More examples

Number	2's Complement	1's Complement
+1	0001	0001
-1	1111	1110
+5	0101	0101
-5	1011	1010
+0	0000	0000
-0	none	1111

Negating a 2's Complement Number

- Invert all the bits
- Then add 1

```
+2 = 0010 (binary) → (invert) →  1101 + 1
     =  1110 (binary) = -2

-2 = 1110 (binary) → (invert) →  0001 +1
     =  0010 (binary) = +2
```

Sign-Extension in 2's Complement

- To make a k-bit number wider
  - simplely replicate the sign bit
  - Example 1

$$110 = -1\times2^2 + 1\times2^1 = -2$$

$$1110 = -1\times2^3 + 1\times2^2 + 1\times2^1 = -2$$

$$11110 = -1\times2^4 + 1\times2^3 + 1\times2^2 + 1\times2^1 = -2$$

Sign-Extension in 2's Complement

- To make a k-bit number wider
  - simplely replicate the sign bit
  - Example 2

$$010 = 1\times2^1 = 2$$
$$0010 = 1\times2^1 = 2$$
$$00010 = 1\times2^1 = 2$$

Addition

- Operation is the same regardless of sign
- No need of end-around carry
- Only the interpretation of the results is different

```
  0101 (+5)        1011 (-5)        0110 (+6)
+0001 (+1)      +1111 (-1)      +1111 (-1)
  ------           ------           ------
  0110 (+6)        1010 (-6)        0101 (+5)
```

Subtraction

- Subtraction, A – B = A + (-B)
  - First, negating the number
  - Then, add the negative number,

2's Complement Features

- Addition and subtraction require no pre-calculation decisions (unlike Sign-Magnitude)
- Addition and subtraction require no end-around carry
- No two zeroes problem

Two's Complement Overflow

- All representations can overflow
  - If both inputs to an addition or a subtraction have the same sign,
  - and if the output sign is different,
  - then overflow has occurred.
- Let's focus on 2's complement here

when the arithmetic result exceeds the fixed number of bits available,

```
  1011  (-5)
+ 1100  (-4)
------------

 0111  (+7) ??
```

Two's Complement Overflow

- First, sign-extend the operands
- Then, do the addition

```
  1011  (-5)
+ 1100  (-4)
-------------
 10111  (-9)
```

When Can Overflow Occur?

- Adding two positive numbers - yes
- Adding two negative numbers - yes
- Adding a positive to a negative - no
- Adding a negative to a positive - no

General Handling of Overflow

- Adding two k-bit numbers gives a k+1 bit result
- Two options include:
  - Sign extend and keep the entire k+1 bit result
  - Throw away the new bit generated (keep k bits)
    - Detect overflow and signal an error
    - Ignore the overflow (this is what most systems do)
- The choice is up to the designer

Binary Arithmetic Comparison

	Sign-Magnitude	**One's Complement**	**Two's Complement**
Negative Number	Flip the sign bit	Complement all the bits	Complement all bits and add 1
Zeroes	2 Zeroes (+0, -0)	2 Zeroes (+0, -0)	1 Zero
Largest Number	Same number of + and – numbers	Same number of + and – numbers	One more negative number than positive
Logic Required	Requires Adder and Subtracter; extra logic to identify larger operand, compute sign, etc.	Only Adder required; carry wraps around (end-around carry)	Only Adder required
Overflow Detection	N/A (handled by sign/magnitude logic)	Overflow: carry from high-order adder bit is wrapped around	Overflow: sign of both operands is the same and sign of sum differs

Use of Two's Complement

- In C and in java
  - byte, 8-bit: integers from -2^{8-1} to $2^{8-1}-1$.
  - short ,16-bit: integers from 2^{16-1} to $2^{16-1}-1$.
  - int, 32-bit: integers from -2^{32-1} to $2^{32-1}-1$.

Floating Point Numbers

- It is not always possible to express numbers in integer form.
- Real, or **floating point** numbers are used in a computer when:
  - The number to be expressed is outside of the integer range of the computer, like 5.375×10^{25}
  - Or, when the number contains a fraction, like 345.0256

Fixed Point

- **Idea:** Like in base 10, let's add binary decimal places to our existing number representation.

5	9	3	4	.	2	1	6
10^3	10^2	10^1	10^0		10^{-1}	10^{-2}	10^{-3}

1	0	1	1	.	0	1	1
2^3	2^2	2^1	2^0		2^{-1}	2^{-2}	2^{-3}

Fixed Point

- **Problem:** we have to fix where the decimal point is in our representation. What should we pick? This also fixes us to 1 place per bit.

. 0 1 1 0 0 1 1

1/2s 1/4s 1/8s …

1 0 1 1 0 . 1 1

16s 8s 4s 2s 1s 1/2s 1/4s

Fixed Point

- **Problem:** we have to fix where the decimal point is in our representation. What should we pick? This also fixes us to 1 place per bit.

•Base 10 •Base 2

•5.07E30 = •10$\underbrace{\ldots\ldots}_{\text{•100 zeros}}$0.1

•9.86E-32 = •0.0$\underbrace{\ldots\ldots}_{\text{•100 zeros}}$01

•To be able to store both these numbers using the same fixed point representation, the bitwidth of the type would need to be at least 207 bits wide!

Let's Get Real

Would be nice to have in a real number representation

- Represent widest range of numbers possible
- Flexible "floating" decimal point
- Scientific notation , e.g. 1.2×10^6
- Still be able to compare quickly
- Have more predictable over/under-flow behavior

Normalisation of floating point numbers

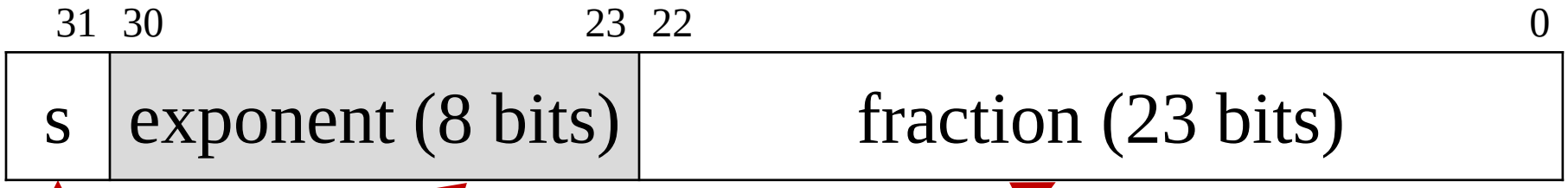
- To maximise the precision for a given number of digits, numbers are, usually, stored **with no leading zeros**.
- Process of transformation the numbers into such a form is called **normalisation**.
- Example.
  - A number: 0.00123×10^7
  - Its normalised form: 1.23×10^4

IEEE Floating Point

Let's aim to represent numbers of the following scientific-notation-like format:

$$x * 2^{y}$$

Single Precision Floating Point



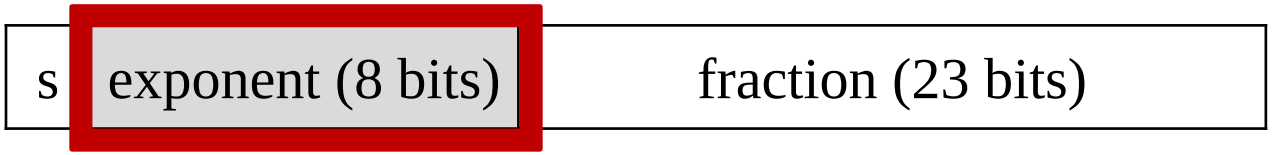
- Sign bit

(0 = positive)

(1 = negative)

$$x * 2^{y}$$

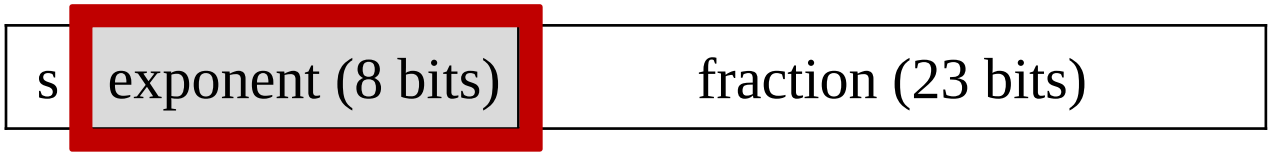
Exponent



Exponent (Binary)	Exponent (Base 10)
11111111	255
11111110	254
11111101	253
11111100	252
…	?
00000011	3
00000010	2
00000001	1
00000000	0

- Suppose x = 1.5 (decimal) or 1.1(binary), what is the range of $x * 2^y$

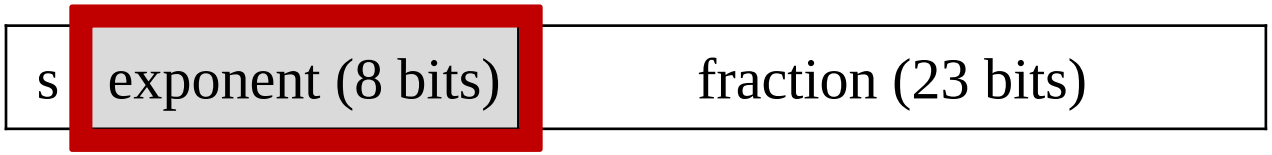
Exponent



Exponent (Binary)	Exponent (Base 10)
11111111	RESERVED
11111110	127
11111101	126
11111100	125
…	…
00000011	-124
00000010	-125
00000001	-126
00000000	RESERVED

- More reasonable to set up an excess of 127

Exponent



- The exponent is **not** represented in 2's complement,
- Instead, exponents are sequentially represented starting from 000…1 (most negative) to 111…10 (most positive) with an excess of 127

11111110	254 – 127 = 127
11111101	253 – 127 = 126
…	…
00000010	2 – 127 = -125
00000001	1 – 127 = -126

Fraction



$$x * 2^{y}$$

- We could encode whatever x is in the fraction field. But there's a trick we can use to make the most out of the bits we have.

An Interesting Observation

In Base 10:

$42.4 \times 10^5 = 4.24 \times 10^6$

$324.5 \times 10^5 = 3.245 \times 10^7$

$0.624 \times 10^5 = 6.24 \times 10^4$

- We tend to adjust the exponent until we get down to one place to the left of the decimal point.

In Base 2:

$10.1 \times 2^5 = 1.01 \times 2^6$

$1011.1 \times 2^5 = 1.0111 \times 2^8$

$0.110 \times 2^5 = 1.10 \times 2^4$

- **Observation:** in base 2, this means there is *always* a 1 to the left of the decimal point!

Fraction



$$x * 2^{y}$$

- We can adjust this value to fit the format described previously. Then, x will always be in the format **1.XXXXXXXXX…**
- Therefore, in the fraction portion, we can encode

Discussion

- Is it possible to represent *all* real numbers using a fixed-width representation?

Float Converter

- https://www.h-schmidt.net/FloatConverter/IEEE754.html

Floats and Graphics

- https://www.shadertoy.com/view/4tVyDK

Let's Get Real

What would be nice to have in a real number representation?

- Represent widest range of numbers possible
- Flexible "floating" decimal point
- Represent scientific notation numbers, e.g. 1.2×10^6
- Still be able to compare quickly
- Have more predictable over/under-flow behavior

Representing Zero

- The floating representation of zero is all zeros (with any value for the sign bit)
  - This means there are two representations for zero!

Sign	Exponent	Fraction
any	All zeros	All zeros

Representing Small Numbers

If the exponent is all zeros, we switch into "denormalized" mode.

Sign	Exponent	Fraction
any	All zeros	Any

- We now treat the exponent as -126, and the fraction as *without* the leading 1.
- This allows us to represent the smallest numbers as precisely as possible.

Representing Exceptional Values

If the exponent is all ones, and the fraction is all zeros, we have +- infinity.

Sign	Exponent	Fraction
any	All ones	All zeros

- The sign bit indicates whether it is positive or negative infinity.
- Floats have built-in handling of over /underflow!
  - Infinity + anything = infinity
  - Negative infinity + negative anything = negative infinity

Representing Exceptional Values

If the exponent is all ones, and the fraction is nonzero, we have **Not a Number.**

Sign	Exponent						Fraction
any	1	…	…	…	…	1	Any nonzero

- NaN results from computations that produce an invalid mathematical result.
  - Sqrt(negative)
  - Infinity / infinity
  - Infinity + -infinity
  - Etc.

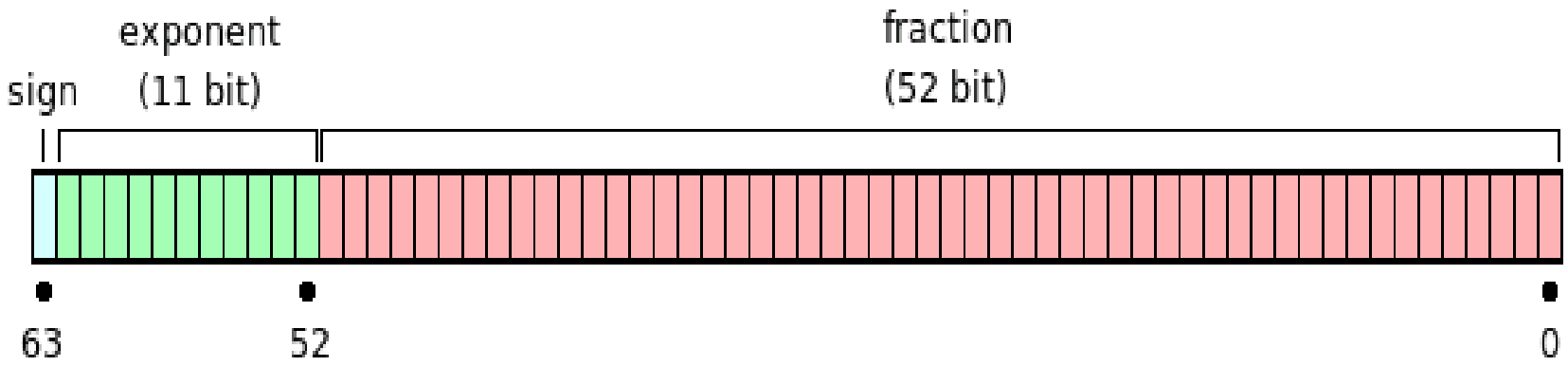
Number Ranges

- 32-bit integer (type int) range: 2,147,483,648 to 2,147,483,647
  - Every integer in that range can be represented
- 32-bit floating point (type float) range: 1.2 $*10^{-38}$ to ~3.4 $*10^{38}$
  - Not all numbers in the range can be represented (not even all integers in the range can be represented!)
  - Gaps can get quite large! (larger the exponent, larger the gap between successive fraction values)

IEEE standard 754 (cont.)

- **Double-precision floating point format**:
  - 64-bits: 1 bit for sign, 11 bits of exponent, 52 of mantissa.
  - The exponent is formatted using **excess-1023** notation.
  - Overall, the standard allows approximately 15 decimal digit precision and approximate value range 10^{-324} to 10^{308}.

Double-precision 64 bit IEEE 754



- Q. What are the pros & cons of BCD?

- Q. Is it possible to have a 3’s complement scheme?

- Q. Can overflow still occur under 2’s complement addition?

- Q. What are the pros & cons of 2’s complement?

- Q. What's the 7-bit two’s complement representation of -17?

- Q. We have learned the representation of base 2 number (binary) and base 16 number (hexdecimal). Now we have a base 4 number 231. What is the decimal representation of the number?